॥ श्रीगणेशाय नमः ॥

Asymptotic Analysis of Algorithms

Algorithms

An algorithm is a step-by-step procedure for solving a problem or accomplishing a task. It is a sequence of well-defined instructions or rules that specify how to solve a problem in a finite # of steps.

Key characteristics of an algorithm:

- Finite: It should terminate after a finite # of steps.
- Unambiguous: Every step should clearly specify the next step.
- I/O: An algo takes an input and produces an output.
- Effectiveness: An algo should solve a problem correctly for all possible inputs within a reasonable amount of time.

Asymptotic analysis

It is a method in computer science and mathematics to describe the limiting behavior of a function or an algorithm as its input size tends towards infinity.

It checks how the time and space complexities of an algo behave for a really large input size ($n \to \infty$).

It helps us understand how the performance of an algorithm scales with increasing input size.

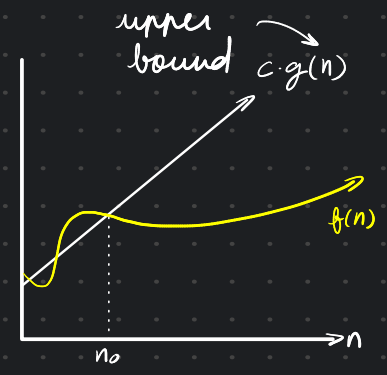
Big-Oh notation:

$f(n) = O(g(n))$,

if there exists a positive constant c and some n_o such that

$f(n) \le c*g(n) \quad \forall\ n > n_o$

$f(n) \le c \cdot g(n)$



E.g. $f(n) = n^2 \quad g(n) = n^3$

$n^2 \le c \cdot n^3$

$c = 1$ satisfies this for all $n > n_o \quad n^2 = O(n^3)$

E.g. $f(n) = 2n^2 + 3 \quad g(n) = n^2$

$2n^2 + 3 \le c \cdot n^2$

$c = 10000, \quad n_o = 1000$

$2n^2 + 3 = O(n^2)$

When we have an asymptotic upper bound, we can use the O-notation.

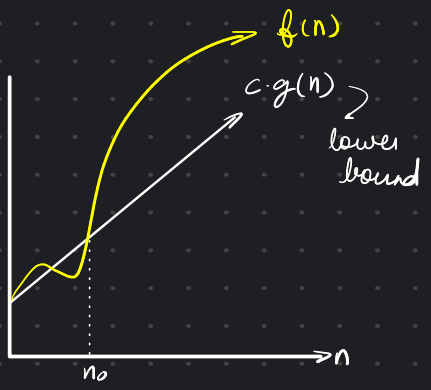
Big-Omega notation (Ω):

$f(n) = \Omega(g(n))$,

if there exists a positive constant c and some n_o such that

$f(n) \ge c*g(n) \quad \forall\ n > n_o$

$f(n) \ge c \cdot g(n)$



E.g. $f(n) = n^3 \quad g(n) = n^2$

$n^3 \ge c \cdot n^2$

$c = 1 \quad n_o = 1$

$n^3 = \Omega(n^2)$

2. $f(n) = 3n^3 + 2 \quad g(n) = n^3$

$3n^3 + 2 \ge c \cdot n^3$

$c = 1$

$\Rightarrow 3n^3 + 2 \ge n^3$

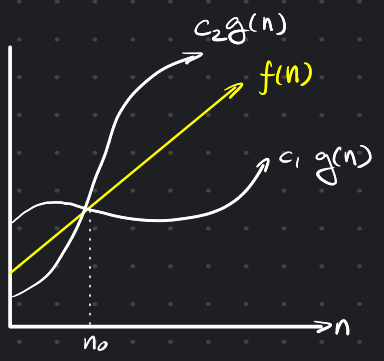
$2n^3 \ge -2$

$n^3 \ge -\frac{2}{3} \quad n_o = -\sqrt[3]{\frac{2}{3}}$

NOTE: When the functions differ only by constants in the leading term, like in $3n^2+2n+1$ and n^2, they are asymptotically the same. We can adjust c and n_o to get O, Ω, and Θ relations like between them.

Big-Theta notation (Θ):

If f(n) = Θ(g(n)), then g(n) is asymptotically tight bound for f(n).



f(n) = Ω(g(n)),
if there exist positive constants c_1, c_2 and some n_0 such that
$c_1 * g(n) \le f(n) \le c_2 * g(n)$ $\forall\ n > n_0$

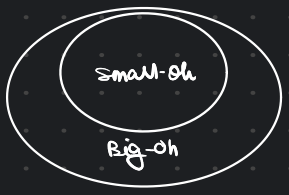
$c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$

E.g. $3n^2 = \Theta(n^2)$
as $2n^2 \le 3n^2 \le 4n^2$

$f(n) = \Theta(g(n))$ (tightly bound)
if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$

Small-oh notation:

The asymptotic upper bound O-notation may or may not be asymptotically tight.
The small o-notation denotes an upper bound that is not asymptotically tight.



$f(n) = o(g(n))$ if for any the constant c, there exists an n_0 s.t. $f(n) < c \cdot g(n)$.

E.g. $f(n) = n$ $\quad g(n) = n \log n$
$f(n) = o(g(n))$
$\Rightarrow f(n) < cg(n)$ as opposed to $f(n) \le c \cdot g(n)$

For $f(n) = O(g(n))$,
n_0 for one value of c is acceptable
for $f(n) \le c \cdot g(n)$

$3n^3 = O(n^3)$ (diff is constant)

For $f(n) = o(g(n))$,
n_0 values must exist for all c values
for $f(n) \le c \cdot g(n)$

$3n^3 \quad n^3$
fails for $c=1$ as $3n^3 < n^3$ doesn't have a n_0
$3n^3 \ne o(n^3)$

But if the difference is not just a constant,
$3n^3 = o(n^3 \log n)$ as $3n^3 < c \cdot (n^3 \log n)$
is satisfied. $\exists n_0\ \forall c$.

Small omega (ω):

The small ω-notation denotes a lower bound that is not asymptotically tight.

$f(n) = \omega(g(n))$
if $\forall\ c > 0\ \ \exists\ n_0 > 0$ s.t. $f(n) > g(n)$

NOTE:
$f(n) = \omega(g(n)) \Rightarrow \lim_{n\to\infty} \frac{g(n)}{f(n)} = 0$

For o-notation,
$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

E.g. $g(n) = n^2 + 2 \quad f(n) = n^3$.

$$\lim_{n\to\infty} \frac{g(n)}{f(n)} = \lim_{n\to\infty} \frac{n^2+2}{n^3} = \lim_{n\to\infty} \frac{\frac{1}{n} + \frac{2}{n^3}}{1} = 0$$

Complexity classes

1. Decreasing functions

   $O(\frac{1}{n}), O(\frac{1}{n^2}), \ldots$

2. Constant functions

   $O(1), O(10,000)$

3. Logarithmic functions

   $O(\log n)$

4. Polynomial functions

   $O(n) < O(n^2) < O(n^3)$

5. Exponential functions

   $O(e^n)$

Q. $\log n$ and $\sqrt{n}$, which grows faster?

→ $\log n$ $\sqrt{n}$

Apply log,

$\log \log n$ $\frac{1}{2}\log n$ ↳ ignore

$\log n > \log \log n$

⇒

$\sqrt{n} = O(\log n)$

Q. $n^{3/2}$ $n \log n$

→ Don't apply logs directly.
Cancel out factors if there are any.

$n^{3/2}$ $n \log n$

$n^{\frac{1}{2}}$ ✗ ✗ $\log n$

$\sqrt{n}$ > $\log n$

$\log n = O(\sqrt{n})$

So $n \log n = O(n^{3/2})$

Q. 2^n n^n

$2 \cdot 2 \cdot 2 \cdots 2$ (n times) $n \cdot n \cdot n \cdots n$ (n times)

$2^n \leq n^n$ for $n_0 \geq 3$ → the difference is not by constant.

$2^n = o(n^3)$

$\Rightarrow 2^n = O(n^3)$

If $f(n) = o(g(n))$
we can extend it to $f(n) = O(g(n))$
(but not the other way around).

Q $n!$ n^n

→ $1 \cdot 2 \cdot 3 \cdots n$ $n \cdot n \cdot n \cdots n$

$n! = o(n^n)$

$\Rightarrow n! = O(n^n)$

Q. 2^n or $n!$

→ $2 \cdot 2 \cdot 2 \cdots 2$ $n(n-1)(n-2) \cdots 2 \cdot 1$

$n! > 2^n$ for $n > 3$

(diff not by constant)

$2^n = o(n!)$

$\Rightarrow 2^n = O(n!)$

For a direct comparison, compare the functions for really large n values.

Q. $\log_2 n$ and $\log_3 n$.

$$\log_3 n = \frac{\log_2 n}{\log_2 3} \leftarrow \text{constant.}$$

(small o fails)

$$\log_3 n = \frac{\log_2 n}{1.5}$$

$$\log_2 n \leq c \frac{\log_2 n}{1.5}$$

fails for $c = 1$ (not all c values have an n_0)

$\log_2 n = O(\log_3 n)$

Also, $\log_2 n = \Omega(\log_3 n)$

$\Rightarrow \log_2 n = \Theta(\log_3 n)$

Here o and ω notations cannot be used.

→ as $\log_2 n \geq \frac{c}{1.5} \log n$ for some c, n_0 ($c = 1.5$)

If $f(n) = \Theta(g(n))$ then $f(n) = O(g(n))$ and $f(n) = \Omega g(n)$.

If $f(n) = o(g(n))$ then $f(n) = O(g(n))$ (not the other way around).

Q. 2^n 3^n

Is it difference by constant?

$2 \cdot 2 \cdot 2 \cdots 2$ (n times) $\quad 3 \cdot 3 \cdot 3 \cdots 3$ (n times)

Don't apply log directly.

$\log 2^n$ $\quad \log 3^n$

$n \log 2$ $\quad n \log 3$

→ constant difference

But this is wrong.

2^n $\quad (2 \times 1.5)^n$

2^n $\quad = 2^n (1.5)^n$

1 $\quad (1.5)^n$

It is not difference by a constant.

$2^n = o(3^n)$ $\quad 2^n \neq \omega(3^n)$

$\Rightarrow 2^n = O(3^n)$ $\quad$ and $2^n \neq \Theta(3^n)$

Q. $(\log n)^{1000000000}$ $\quad n$ $\quad (\log n)^k = o(n)$ $= O(n)$

→ log on both sides,

$10^9 \log \log n < \log n$

(10^9 → ignore)

$n > (\log n)^{10^9}$ not by a constant.

Q. $\log n^{(\log n)}$ $\quad$ n

→ $\cancel{\log n}\ \log\log n \quad \cancel{\log n}$

$\log\log n > 1$

$n < (\log n)^{(\log n)}$

$n = o((\log n)^{(\log n)})$

$= O((\log n)^{(\log n)})$

Q. n^2 $\quad$ n^3

→ Don't apply logs directly.

n^2 $\quad$ $\cancel{n^2} \cdot n$

1 $\quad$ n

$n^2 = o(n^3)$

$\Rightarrow n^2 = O(n^3)$

↳ diff. is not constant

$n^2 \neq \omega(n^3)$

$2\cancel{\log n}$ $\quad$ $3\cancel{\log n}$ $\quad$ WRONG ✗✗

1 $\quad$ 1

$n^2 = O(n^3)$ } $n^2 = \Theta(n^3)$

$n^2 = \Omega(n^3)$ }

Order of common functions

Notation	Name	Example
$O(1)$	constant	Finding the median value for a sorted array of numbers; Calculating $(-1)^n$; Using a constant-size lookup table
$O(\alpha(n))$	inverse Ackermann function	Amortized complexity per operation for the Disjoint-set data structure
$O(\log \log n)$	double logarithmic	Average number of comparisons spent finding an item using interpolation search in a sorted array of uniformly distributed values
$O(\log n)$	logarithmic	Finding an item in a sorted array with a binary search or a balanced search tree as well as all operations in a binomial heap
$O((\log n)^c)$ $c > 1$	polylogarithmic	Matrix chain ordering can be solved in polylogarithmic time on a parallel random-access machine.
$O(n^c)$ $0 < c < 1$	fractional power	Searching in a k-d tree
$O(n)$	linear	Finding an item in an unsorted list or in an unsorted array; adding two *n*-bit integers by ripple carry
$O(n \log^* n)$	*n* log-star *n*	Performing triangulation of a simple polygon using Seidel's algorithm, where $\log^*(n) = \begin{cases} 0, & \text{if } n \leq 1 \\ 1 + \log^*(\log n), & \text{if } n > 1 \end{cases}$
$O(n \log n) = O(\log n!)$	linearithmic, loglinear, quasilinear, or "*n* log *n*"	Performing a fast Fourier transform; fastest possible comparison sort; heapsort and merge sort
$O(n^2)$	quadratic	Multiplying two *n*-digit numbers by schoolbook multiplication; simple sorting algorithms, such as bubble sort, selection sort and insertion sort; (worst-case) bound on some usually faster sorting algorithms such as quicksort, Shellsort, and tree sort
$O(n^c)$	polynomial or algebraic	Tree-adjoining grammar parsing; maximum matching for bipartite graphs; finding the determinant with LU decomposition
$L_n[\alpha, c] = e^{(c+o(1))(\ln n)^\alpha (\ln \ln n)^{1-\alpha}}$ $0 < \alpha < 1$	L-notation or sub-exponential	Factoring a number using the quadratic sieve or number field sieve
$O(c^n)$ $c > 1$	exponential	Finding the (exact) solution to the travelling salesman problem using dynamic programming; determining if two logical statements are equivalent using brute-force search
$O(n!)$	factorial	Solving the travelling salesman problem via brute-force search; generating all unrestricted permutations of a poset; finding the determinant with Laplace expansion; enumerating all partitions of a set

$\equiv O(n^n)$

Q. $1000\, n\log n = O\left(\frac{n\log n}{1000}\right)$. True/false

→ $1000\, n\log n \le c \cdot n\log n$ (c = 10000000) True

NOTE: Whenever leading terms of functions differ only by constants, Ω, O, and Θ can be applied.

E.g. $3n^2+2n+1 = O(n^2)$
$3n^2+2n+1 = \Omega(n^2)$
$3n^2+2n+1 = \Theta(n^2)$
} tight

Also, $3n^2+2n+1 = O(n^3)$
$3n^2+2n+1 = \Omega(n)$
} loose

Tight upper bound means we cannot go below it.
Tight lower bound means we can't go above that.

Q. compare $\sqrt{\log n}$, $\log\log n$

→ $(\log n)^{1/2}$ $\log\log n$

$\frac{1}{2}\log\log n > \log\log\log n$ (×)

$\sqrt{\log n} > \log\log n$ Not different by a constant.

$\sqrt{\log n} = \omega(\log\log n)$
$= \Omega(\log\log n)$

Q. compare 2^{n+1} 2^n

→ $2^n \cdot 2$ 2^n (2 → ignore)

Differs by a constant.

$2^{n+1} = O(2^n)$
$= \Omega(2^n)$
$= \Theta(2^n)$

Q. 2^{2n} 2^n

→ 2^{2n} 2^n

$2^n 2^n$ 2^n

$2^n > 1$

$2^{2n} = \omega(2^n)$
$= \Omega(2^n)$

Q. $\frac{1}{n}$. 1

→ $\frac{1}{n} = o(1)$
$= O(1)$

As $n \to \infty$, $\frac{1}{n} \to 0$

Q. 1,00,000 1

→ Differs by a constant.

NOTE: All constants are asymptotically the same.

$1,00,000 = O(1)$
$= \Omega(1)$
$= \Theta(1)$

Q. $n^{\log n}$ $(\log n)^n$

→ $\log n \log n$ $n\log\log n$

Substitute a large n.

let $n = 2^{1024}$

$\log 2^{1024}$ $\log 2^{1024}$ 2^{1024} $\log\log 2^{1024}$

$(1024)^2$ $2^{1024} \cdot 10$

2^{20} < $2^{1024} \cdot 10$

exponential > polynomial

Q. n^c c^n c: constant

→ $c\log n$ $n\log c$ (× ×)

$\log n < n$

$n^c = O(c^n)$
$= O(cn)$

Q. True/false

a) $f(n) = O(f(n))$ T

b) $f(n) = \Omega(f(n))$ T

c) $f(n) = \Theta(f(n))$ T

d) $f(n) = O(f(\frac{n}{2}))$

e.g. $f(n) = n^2+2$ $f(\frac{n}{2}) = \frac{n^2}{4}+2$ True for this case.

e.g. $f(n) = 2^{2n}$ $f(\frac{n}{2}) = 2^n$ $f(n) \neq O(f(n/2))$

$2^n 2^n$ 2^n

2^n 1 False for this case.

$f(n)$ is NOT always $O(f(n/2))$.

Q. $(n+3)^{25} = O(n^{25})$. True?

→ True.

$(n+3)^{25} \leq c \cdot n^{25}$

$\Rightarrow \frac{(n+3)^{25}}{n^{25}} \leq c$

$\Rightarrow \left(\frac{n+3}{n}\right)^{25} \leq c$

$\Rightarrow \left(1+\frac{3}{n}\right)^{25} \leq c$

As $n \to \infty$, $\left(1+\frac{3}{n}\right) \to 0 \leq c \quad \forall\ c > 0$

Q. $(n+3)^{25} = \Omega(n^{25})$

→ $(n+3)^{25} \geq c \cdot n^{25}$

$\left(\frac{n+3}{n}\right)^{25} \geq c$

$\left(1+\frac{3}{n}\right)^{25} \geq c$

Q. $f(n) = O(f(n^2))$

→ case: $f(n) = n$, $f(n^2) = n^2$ True for this case

case: $f(n) = \frac{1}{n}$, $f(n^2) = \frac{1}{n^2}$ False.

False

NOT always true [decreasing functions].

Q. $f(n) = O((f(n))^2)$ where $f(n) > 1$

→ True.

$f(n)$ is increasing func^n.

$f(n) = O[(f(n))^2]$ ✓

Q. $f(n) = O((f(n))^2)$ where $f(n) < 1$.

→ False

$f(n)$ is a decreasing func^n.

---

*Q. If $f(n) = O(g(n))$ then $2^{f(n)} = O(2^{g(n)})$.

→ False.

let $f(n) = 2n$ $g(n) = n$

2^{2n} 2^n

$2^n 2^n$ 2^n

2^n 1

$2^n \neq O(1)$

NOT always true.

Q. $f(n) = \begin{cases} n^3 & \text{if } 0 < n < 100 \\ n^5 & \text{if } n \geq 100 \end{cases}$ $g(n) = \begin{cases} n^9 & 0 < n < 10{,}000 \\ n^3 & n \geq 10{,}000 \end{cases}$

compare $f(n)$ and $g(n)$.

→ In such cases, we check when all the functions stabilize.

Here, on $n \geq 10{,}000$,

$f(n) = n^5$ $g(n) = n^3$

$f(n) = \omega(g(n))$

$= \Omega(g(n))$

Properties of asymptotic notations

① Reflexivity:

$f(n) = O\ f(n)$

O	Ω	Θ	o	ω
✓	✓	✓	×	×

② Symmetry:

If $f(n) = \Theta\ g(n)$ then $g(n) = \Theta\ f(n)$.

O	Ω	Θ	o	ω
×	×	✓	×	×

③ Transitivity:

If $f(n) = O\ g(n)$ and $g(n) = O\ h(n)$
then $f(n) = O\ h(n)$

O	Ω	Θ	o	ω
✓	✓	✓	✓	✓

④

If $f(n) = O\ g(n)$ and $h(n)$ is a +ve func^n,
then
$f(n) \cdot h(n) = O\ (g(n) \cdot h(n))$

O	Ω	Θ	o	ω
✓	✓	✓	✓	✓

⑤

If $f(n) = O\ g(n)$ and $d(n) = O\ (e(n))$,
then
$f(n) + d(n) = O\ (g(n) + e(n))$
$f(n) \cdot d(n) = O\ (g(n) \cdot e(n))$

O	Ω	Θ	o	ω
✓	✓	✓	✓	✓

Algorithms → iterative / recursive

Analyzing iterative programs

```
main(){
    a = b + c;
}
```

$T = O(1)$
space $= O(1)$

```
main() {
    for (int i=0; i<n; i+=1) {
        num += 1;
    }
}
```

} runs n times

$T = O(n)$
space $= O(1)$

```
for (int i = 0; i < n; i+=2) {
    num += 1;
}
```

method:

Say the loop executes for k iterations and fails for the $(k+1)^{th}$ iteration.

iteration	1	2	3	4	...	k	$k+1$
i	0	2	4	6	...	$2(k-1)$	$2k$

loop fails when $i = n$

$2k = n$

$k = \frac{n}{2}$ (# of iterations)

$k = \lceil \frac{n}{2} \rceil$

So the loop runs $\lceil \frac{n}{2} \rceil$ times

$\frac{n}{2} = O(n)$
$= \Omega(n)$
$= \Theta(n)$

} But mostly it's written $O(n)$.

$T(n) = O(n)$

```
for (int i=1; i < n; i*=2) {
    num += 1;
}
```

→ Let # of iterations $= k$, fails at $(k+1)^{th}$.

iteration	1	2	3	4	...	k	$k+1$
i	1	2	4	8		2^{k-1}	2^k

$2^k \geq n$

$k \geq \log_2 n$

$k = \lceil \log_2 n \rceil$

$T = O(\log_2 n)$

```
for (int i=n; i>2; i = sqrt(i)) {
    //
}
```

iteration	1	2	3	4	...	k	$k+1$
i	n	$\sqrt{n}$	$n^{1/4}$	$n^{1/8}$	...	$n^{\frac{1}{2^{k-1}}}$	$n^{\frac{1}{2^k}}$

$n^{\frac{1}{2^k}} \leq 2$

$\frac{1}{2^k} \log n \leq \log 2$

$2^k \leq \frac{\log n}{\log 2}$

$2^k \leq \log_2 n$

$k \leq \log_2 \log_2 n$

$k = \lfloor \log_2 \log_2 n \rfloor$

$T(n) = O(\log \log n)$

Q.

```
int i=n;
while (i>1) {
    num += 1;
    i = i/2;
}
```

→

iteration	1	2	3	...	K	K+1
i	n	$\frac{n}{2}$	$\frac{n}{2^2}$	...	$\frac{n}{2^{K-1}}$	$\frac{n}{2^K}$

$\frac{n}{2^k} \leq 1$

$n \leq 2^k$

$k \geq \log_2 n$

$k = \lceil \log_2 n \rceil$

$T(n) = O(\log n)$

```
for (int i=1; sq(i) < n; i++) {
    //
}
```

→ $T(n) = O(\sqrt{n})$

```
for (int i=n/2; i<n; i*=2) {
    //
}
```

→

1	2
i	n
$\frac{n}{2}$	

↳ It'll stop here itself.

$T = O(1)$.

```
for (int i=0; i<n; i++) {
    for(int j=0; i<m; j++) {
        num += 1;
    }
}
```

Unrolling the loop.

$i = 0$	$i = 1$	$i = 2$	$i = n-1$
$j = 0, 1, \ldots, m-1$	$j = 0, 1, \ldots, m-1$	$j = 0, 1, \ldots, m-1$	$j = 0, \ldots, m-1$

$T = O(m * n)$

```
i = 1;
while (i<n) {
    int j=1;
    while (j<n) {
        //
        j *= 2;
    }
    i *= 2;
}
```

$\log_2 n$ $\log_2 n$

$T(n) = O(\log n \log n)$

```
for (int i = n*n; i>1; i = i/2) {
    //
}
```

	1	2	3	4	...	K	K+1
	n^2	$\frac{n^2}{2}$	$\frac{n^2}{4}$	$\frac{n^2}{8}$	...	$\frac{n^2}{2^{K-1}}$	$\frac{n^2}{2^K}$

$\frac{n^2}{2^k} \leq 1$

$n^2 \leq 2^k$

$k \geq \log_2 n^2$

$k \geq 2 \log_2 n$

$T(n) = O(\log n)$

Space $= O(1)$

```
for (int i=n²; i>1; i=i/2) {
    for (int j=1; j<n³; j++){
        num += 1;
    }
}
```

$O(n^3)$ × $O(\log n)$ $\quad T(n) = O(n^3 \log n)$

```
for (i=n/2; i<=n; i++) {        ← O(n)
    for (j=1; j<=n/2; j++){     ← O(n)
        for (k=1; k<=n; k=k*2){ ← O(log n)
            printf("Hello there");
        }
    }
}
```

$T(n) = O(n^2 \log n)$

```
int i=1, s=1;
while (s<=n) {
    i+=1;
    s+=i;
    //
}
```

iteration #	0 (before loop)	1	2	3	...	k
i	1	2	3	4		k+1
s	1	3 (1+2)	6 (1+2+3)	10 (1+2+3+4)		$1+2+\dots+(k+1)$ $\frac{(k+1)(k+2)}{2}$

$\frac{(k+1)(k+2)}{2} > n$

$k^2 + 3k + 2 > 2n$

ignore all of these

$k^2 > n$

$k = \lceil \sqrt{n} \rceil$

$T(n) = O(\sqrt{n})$

```
int i=1, s=1;
while(s<=n) {
    i+=1;
    s+=i²;
    //
}
```

iteration #	0	1	2	...	k
i	1	2	3		k+1
s	1	1+4	1+4+9		$1+2^2+3^2+\dots+(k+1)^2$

$\approx \frac{(k)(k+1)(2k+1)}{6} > n$

$k^3 > n$

$T(n) = O(\sqrt[3]{n})$

```
p=0;
for (int i=1; i<=n; i*=2) {   ← runs log₂n times, so p = log₂n
    p+=1;
}
for (int j=1; j<p; j*=2) {    ← runs log₂p times i.e. log₂(log₂n)
    //
}
```

$T(n) = O(\lg \lg n + \lg n)$

$= O(\lg n)$

```
for (int i=1; i<=n; i++) {
    j=2;
    while (j<=n) {
        j=j²;
    }
}
```

Outer loop: runs $O(n)$ times

Inner: $O(\lg \lg n)$ times

$T(n) = O(n \lg \lg n)$

iteration	1	2	3	4	...	k	k+1
j	2	4 (2^2)	16 (2^4)	256 (2^8)		$2^{2^{k-1}}$	2^{2^k}

$2^{2^k} > n$

$2^k > \lg n$

$k > \lg \lg n$ $\quad k = O(\lg \lg n)$

Inner loop runs $O(\lg \lg n)$ times.

```
int i=1;
while(i<n) {
    int j=1;
    while(j<n) {
        int k=1;
        while(k<n) {
            //
            k=k*2;
        }
        j=j*2;
    }
    i=i*2;
}
```

← $O(\lg n)$

← $O(\lg n)$

← $O(\lg n)$

$T(n) = O(\lg\lg\lg n)$

```
for (int i=1; i<=n; i+=1) {
    for (int j=1; j<=n; j+=i) {
        //
    }
}
```

$i=1$ $j = 1\ 2\ 3\ \cdots\ n$ ← n times

$i=2$ $j = 1\ 3\ 5\ \cdots\ n$ ← $n/2$ times

$i=3$ $j = 1\ 4\ 7\ \cdots\ n$ ← $n/3$ times

⋮

$i=n$ ← $\frac{n}{n} = 1$ times

$$\left(n + \frac{n}{2} + \frac{n}{3} + \cdots + 1\right)$$

$$n\underbrace{\left(1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n}\right)}_{\log n}$$

$n \log n$

```
int i=2;
while (i<n) {
    i=i²;
}
```

iteration	1	2	3	k+1
i	2	4	16	
	2^1	2^2	2^4	
	2^{2^0}	2^{2^1}	2^{2^2}	2^{2^k}

$2^{2^k} \geq n$

$k = \lg\lg n$

```
int i=25;
while (i<n) {
    i=i³;
}
```

iteration	1	2	3	k+1
i	25	25^3	25^9	
	25^{3^0}	25^{3^1}	25^{3^2}	25^{3^k}

$25^{3^k} \geq n$

$3^k \geq \log_{25} n$

$k \geq \log_3 \log_{25} n$

$T(n) = O(\log_3 \log_{25} n)$

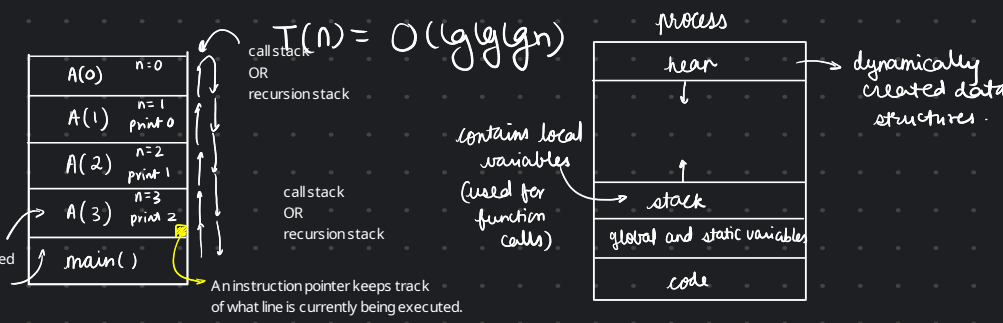
$T(n) = O(\log\log n)$

Recursion

Both iterative and recursive programs are equally powerful. Any iterative program will have a corresponding recursive program and vice versa.

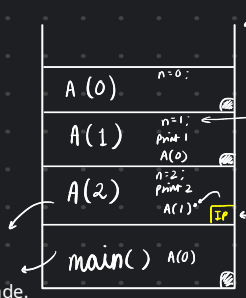
A recursive function is one that calls itself.

```c
A(n) {
    if (n > 0) {
        printf("%d\n", n-1);
        A(n-1);
    }
}
```

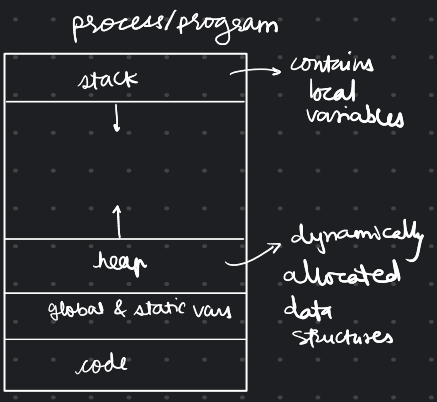


Recursion

```
A(n) {
    if (n > 0) {
        printf("%d\n", n);
        A(n-1);
    }
}
```

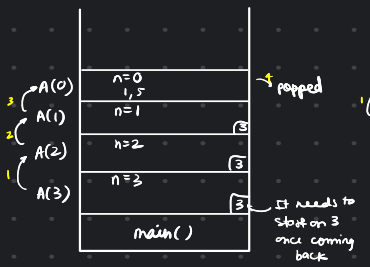


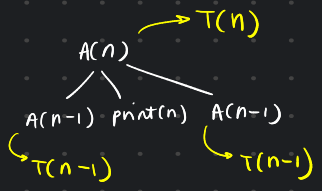
Activation records are pushed into the stack when function calls are made.

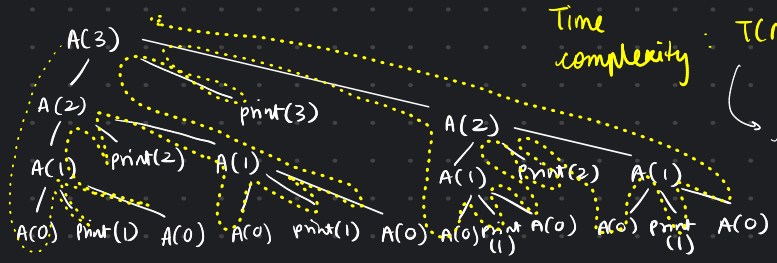


```
1  A(n) {
2      if (n > 0) {
3          A(n-1);
4          printf(n);
5          A(n-1);
       }
   }

   main() {
       A(3);
   }
```







Time complexity

$T(n) = 2T(n-1) + c$

c: constant

Recurrence relation.

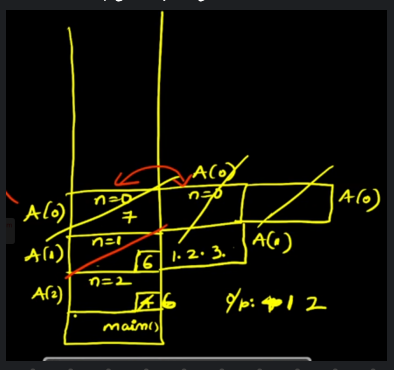
output: 1213121

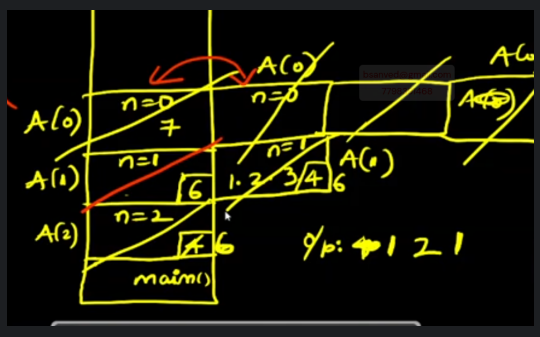
Uy, output for A(5): (A(4))5(A(4))

(A(3)4 A(3))5(A(3)4A(3))

1213121412131215121314121 3121

For A(2)





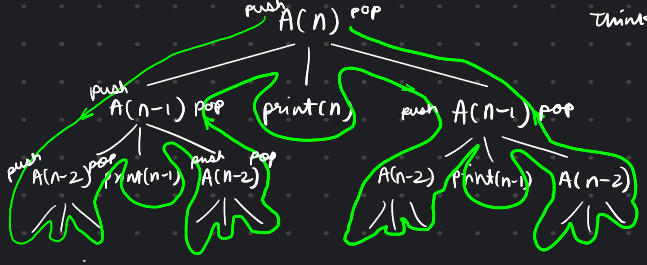
Space =

(3 call stack records)

+

(local variables)

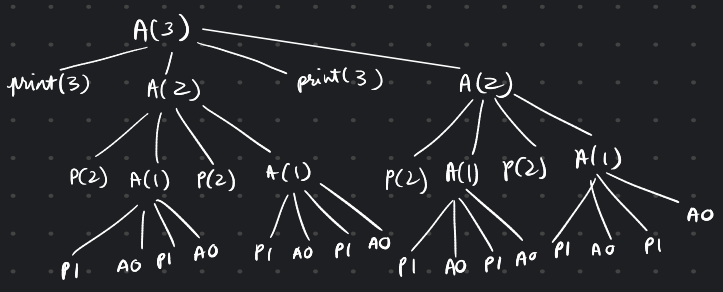
Space complexity: O(n) auxiliary.



Think of a node being pushed into the call stack when it is first visited in the traversal.

```
A(n)
{
    if (n>0)
    {
        print(n)
        A(n-1)
        print(n)
        A(n-1)
    }
}
```

A(3)=?



A(1) : 11

A(2) : 211211

A(3) : 3211211321121 1

$T(n) = 2T(n-1) + c, \quad n > 0$

Recurrence relations

There are 3 common methods to solve recurrence relations:
- Substitution
- Using recursive trees
- Master theorem

When to use these particular methods?

$T(n) = a \cdot T\left(\frac{n}{b}\right) + O(n)$ ← master theorem

$T(n) = T(n-1) + T(n-2) + T(n-3)$ ← tree
↳ more than 1 recursive call

$T(n) = 2T(n-1) + n$ ← substitution
↳ a single term

In the substitution method and recursion tree method, we can find space complexities as well.

Substitution

base/terminating condition

Q. $T(n) = \begin{cases} 1 & \text{if } n=1 \\ T(n-1)+n & \text{if } n>1 \end{cases}$ recursion

→ $T(n) = T(n-1) + n$
$= T(n-2) + (n-1) + n$
$= T(n-3) + (n-2) + (n-1) + n$
$= T(n-4) + (n-3) + (n-2) + (n-1) + n$
⋮
$T(n) = T(n-k) + [n-(k-1)] + [n-(k-2)] + \cdots + (n-2) + (n-1) + n$

$T(n) = T(n-1) + n$ —①
$T(n-1) = T(n-2) + (n-1)$ —②
$T(n-2) = T(n-3) + (n-2)$ —③

this will stop at the base condition $[T(1)=1]$

say $n-k=1$
$\Rightarrow k = n-1$

$T(n) = T(1) + [n-(n-1-1)] + [n-(n-1-2)] + \cdots + (n-2) + (n-1) + n$
$= T(1) + 2 + 3 + \cdots + (n-2) + (n-1) + n$
$= 1 + 2 + 3 + \cdots + n$
$= \frac{n(n+1)}{2}$
$= O(n^2)$
$= \Omega(n^2)$
$= \Theta(n^2)$

Q. $T(n) = \begin{cases} 1 & \text{if } n=0 \\ T(n-1) * n & \text{if } n>0 \end{cases}$

→ $T(n) = T(n-1) \cdot n$
$= T(n-2)(n-1)(n)$
$= T(n-k) \cdot [[n-(k-1)] \cdot [n-(k-2)] \cdots (n-1)\, n]$

let $n-k=0$
$\Rightarrow k=n$

$T(n) = T(0) \cdot [(n-(n-1))(n-(n-2)) \cdots (n-1)(n)]$
$= 1 \cdot [1 \cdot 2 \cdots (n-1)(n)]$
$= n!$

Space complexity $= O(k) = O(n)$

$T(n)$
/
$T(n-1)$
/
.
.
.
$T(n-k)$

Stack size $= O(k) = O(n)$

Q. $T(n) = \begin{cases} 0 & \text{if } n=0 \\ T(n-2)+n^2 & \text{if } n>0 \end{cases}$

→

$T(n) = T(n-2) + n^2$

$= T(n-4) + (n-2)^2 + n^2$

$= T(n-2k) + [(n-(k-2))^2 + (n-(2k-4))^2 + \cdots + (n-2)^2 + n^2]$

let $n-2k=0$

$k=\frac{n}{2}$ Space $= O(k) = O(n)$.

$T(n) = T(0) + [(n-(n-2))^2 + (n-(n-4))^2 + \cdots + (n-2)^2 + n^2]$

$= 0 + 2^2 + 4^2 + \cdots + (n-2)^2 + n^2$ ↳ $= 2^2[1^2+2^2+\cdots+(\frac{n}{2})^2]$

$= \frac{n(n+1)(2n+1)}{6} - n^2$ $= 2^2 \cdot \frac{(\frac{n}{2})(\frac{n}{2}+1)(n+1)}{6}$

$= O(n^3)$ $= O(n^3)$

$= \Omega(n^3)$

$= \Theta(n^3)$

Time $= O(n^3)$, space $= O(n)$.

Q. $T(n) = \begin{cases} 0 & \text{if } n=0 \\ T(n-2)+\log_2 n & \end{cases}$

→ $T(n) = T(n-2) + \lg n$

$= T(n-4) + \lg(n-2) + \lg n$

$= T(n-6) + \lg(n-4) + \lg(n-2) + \lg n$

$= T(n-2k) + \lg(n-(2k-2)) + \lg(n-(2k-4)) + \cdots + \lg(n-2) + \lg(n)$

let $n-2k=0$

$k=\frac{n}{2}$

$T(n) = T(0) + \lg 2 + \lg 4 + \cdots + \lg(n-2) + \lg n$

$= \lg[2 \cdot 4 \cdot \cdots \cdot (n-2) \cdot n]$

$= \lg[(2\cdot1)(2\cdot2)\cdots(2)(\frac{n}{2})]$

$= \lg[2^{n/2} \cdot (\frac{n}{2}!)]$

$= \frac{n}{2}\lg 2 + \lg(\frac{n}{2}!)$ ↳ dominating term

$= O(\lg(\frac{n}{2}!))$

$= O(\log n^n)$

$= O(n \log n)$

Q. $T(n) = \begin{cases} 0 & \text{if } n=1 \\ T(n/2)+c \end{cases}$

→ $T(n) = T(\frac{n}{2}) + c$

$= T(\frac{n}{4}) + 2c$

$= T(\frac{n}{8}) + 3c$

$= T(\frac{n}{2^k}) + kc$

let $\frac{n}{2^k} = 1 \Rightarrow 2^k = n$

$\Rightarrow k = \lg n$ space = $O(\log n)$

$T(n) = 0 + kc$

$= \lg n \cdot c$

$T(n) = O(\lg n)$ time = $O(\log n)$

$= \Omega(\lg n)$

$= \Theta(\lg n)$

Q. $T(n) = \begin{cases} 1 & \text{if } n=1 \\ T(\frac{n}{2})+n & \text{if } n>1 \end{cases}$ → preferably solved using the master theorem.

→ $T(n) = T(\frac{n}{2}) + n$

$= T(\frac{n}{4}) + \frac{n}{2} + n$

$= T(\frac{n}{8}) + \frac{n}{4} + \frac{n}{2} + n$

$= T(\frac{n}{2^k}) + \frac{n}{2^{k-1}} + \cdots + \frac{n}{4} + \frac{n}{2} + n$

let $\frac{n}{2^k} = 1 \Rightarrow k = \lg n$

$T(n) = 1 + \frac{n}{2^{\lg n - 1}} + \cdots + \frac{n}{4} + \frac{n}{2} + n$

$\lg n - 1 = \lg n - \lg 2 = \lg n/2$

$= 1 + \frac{n}{2^{\lg n/2}} + \frac{n}{2^{\lg n/4}} + \cdots + \frac{n}{4} + \frac{n}{2} + \frac{n}{1}$

$= 1 + \frac{n}{n/2} + \frac{n}{n/4} + \cdots + \frac{n}{4} + \frac{n}{2} + \frac{n}{1}$

$= 1 + 2 + 4 + \cdots + \frac{n}{4} + \frac{n}{2} + n$

$= 2^0 + 2^1 + 2^2 + \cdots + 2^k$

↳ GP with (k+1) terms

$= \frac{1(2^{k+1}-1)}{(2-1)}$

$= 2^k \cdot 2 - 1$

$= O(n)$

$= \Omega(n)$

$= \Theta(n)$

$\frac{n}{2^k} = 1 \Rightarrow \frac{n}{2^k \cdot 2^{-1}} = 2$

$\Rightarrow \frac{n}{2^{k-1}} = 2$

$\Rightarrow \frac{n}{2^{k-2}} = 2^2$

$2^0 + 2^1 + 2^2 + \cdots + 2^k$

$= \frac{2^0(1-2^{k+1})}{(1-2)}$

$\approx 2^k \approx n$

$= O(n)$

Q. $T(n) = \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right)+n & \end{cases}$

$\rightarrow \quad T(n) = 2T\left(\frac{n}{2}\right)+n$

$= 2\left[2T\left(\frac{n}{4}\right)+\frac{n}{2}\right]+n$

$= 2^2T\left(\frac{n}{4}\right)+n+n$

$= 2^2\left[2T\left(\frac{n}{8}\right)+\frac{n}{4}\right]+2n$

$= 2^3T\left(\frac{n}{8}\right)+3n$

$= 2^kT\left(\frac{n}{2^k}\right)+kn$

put $\frac{n}{2^k}=1 \Rightarrow k=\lg n$

$T(n) = n + n\lg n$

$= O(n\log n)$

Q. $T(n) = \begin{cases} 8T(\frac{n}{2}) + n^2 \\ 1 \qquad\qquad n=1 \end{cases}$

$\rightarrow T(n) = 8T(n/2) + n^2$

$= 8\left[8T(n/4) + (\frac{n^2}{2})\right] + n^2$

$= 8^2 T(\frac{n}{4}) + 2n^2 + n^2$

$= 8^2\left[8T(\frac{n}{8}) + (\frac{n}{4})^2\right] + 2n^2 + n^2$

$= 8^3 \cdot T(\frac{n}{8}) + 4n^2 + 2n^2 + n^2$

$= 8^k T\left(\frac{n}{2^k}\right) + \left[2^{k-1} + 2^{k-2} + \cdots + 2^1 + 2^0\right] n^2$

let $\frac{n}{2^k} = 1 \quad \Rightarrow k = \lg n$

$T(n) = n^3 \cdot T(1) + \frac{n^2(2^k - 1)}{(2-1)} = n^3(1) + n^2(n-1)$

$= O(n^3)$

Q. $T(n) = \begin{cases} 2 & \text{if } n=2 \\ 7T(\frac{n}{2}) + n^2 & \text{if } n>2 \end{cases}$

$\rightarrow T(n) = 7T(\frac{n}{2}) + n^2$

$= 7\left[7T(\frac{n}{4}) + \frac{n^2}{4}\right] + n^2$

$= 7^2 T(\frac{n}{4}) + \frac{7}{4}n^2 + n^2$

$= 7^2\left[7T(\frac{n}{8}) + \frac{n^2}{16}\right] + \frac{7}{4}n^2 + n^2$

$= 7^3 T(\frac{n}{8}) + \frac{7^2}{4^2}n^2 + \frac{7^1}{4^1}n^2 + \frac{7^0}{4^0}n^2$

$= 7^k T\left(\frac{n}{2^k}\right) + \left[(\frac{7}{4})^0 + (\frac{7}{4})^1 + \cdots + (\frac{7}{4})^{k-1}\right] n^2$

let $\frac{n}{2^k} = 2$

$2^{k+1} = n$

$k+1 = \lg n$

$k = \lg n - 1$

$= 7^{\lg n - 1}(2) + \left[\frac{(\frac{7}{4})^{\lg n - 1} - 1}{\frac{7}{4} - 1}\right] n^2$

$= \frac{7^{\lg n}}{7} \cdot 2 + \frac{(\frac{7}{4})^{\lg n - 1} - 1}{\frac{3}{4}} n^2$

↓ taking only the leading terms

$= n^{\lg 7} + n^2 \frac{7^{\lg n}}{4^{\lg n}}$ → $n^{\lg 4} = n^2$

$= n^{\lg 7} + \frac{n^2}{n^2} 7^{\lg n} = n^{\lg 7} + n^{\lg 7} \approx O(n^{2.81})$

Q $T(n) = \begin{cases} 2T(\frac{n}{2}) + n & n > 1 \\ 1 & n = 1 \end{cases}$

$\rightarrow T(n) = 2T(\frac{n}{2}) + n$

$= 2\left[2T(\frac{n}{4}) + \frac{n}{2}\right] + n$

$= 2^2\left[2T(\frac{n}{8}) + \frac{n}{4}\right] + 2n$

$= 2^3 T(\frac{n}{8}) + 3n$

$= 2^k \cdot T(\frac{n}{2^k}) + kn$

let $n = 2^k$

$T(n) = n \cdot (1) + (\lg n)(n)$

$= n + n\lg n$

$= O(n\lg n)$

Q $T(n) = \begin{cases} 1 & n=1 \\ 2T\left(\frac{n}{2}\right) + n\log n & \text{if } n>1 \end{cases}$

$\rightarrow T(n) = 2T\left(\frac{n}{2}\right) + n\log n$

$= 2\left[2T\left(\frac{n}{4}\right) + \frac{n}{2}\log\frac{n}{2}\right] + n\log n$

$= 4T\left(\frac{n}{4}\right) + n\log\frac{n}{2} + n\log n$

$= 4\left[2T\left(\frac{n}{8}\right) + \frac{n}{4}\log\frac{n}{4}\right] + n\left[\log\frac{n}{2} + \log n\right]$

$= 8T\left(\frac{n}{8}\right) + n\left[\log\frac{n}{4} + \log\frac{n}{2} + \log n\right)$

$\log n + \log\frac{n}{2} + \log\frac{n}{4}$

$= \log n + \log n - \log 2 + \log n - \log 4$

$= 3\log n - [\log 2^0 + \log 2^1 + \log 2^2]$

$= 2^k T\left(\frac{n}{2^k}\right) + n\left[k\log n - (\log 2^0 + \log 2^1 + \cdots + \log 2^{k-1})\right]$

$= 2^k T\left(\frac{n}{2^k}\right) + n\left[k\log n - (0 + 1 + 2 + \cdots + k-1)\right]$

$= 2^k T\left(\frac{n}{2^k}\right) + n\left[k\log n - \frac{(k-1)(k)}{2}\right]$

let $n = 2^k$

$= n + n\left[\log n \log n - \frac{(\log n - 1)(\log n)}{2}\right]$

$(\log n)^2$

$= O(n(\log n)^2)$

Q. $T(n) = \begin{cases} 1 & \text{if } n=1 \\ 2T(n-1)+n & n>1 \end{cases}$

→

$T(n) = 2T(n-1)+n$

$= 2\left[2T(n-2)+(n-1)\right]+n$

$= 2^2 T(n-2) + 2(n-1)+n$

$= 2^2\left[2T(n-3)+(n-2)\right] + 2(n-1)+n$

$= 2^3 T(n-3) + 2^2(n-2)+2(n-1)+n$

$= 2^k T(n-k) + \left[2^{k-1}(n-(k-1)) + 2^{k-2}(n-(k-2)) + \dots + 2^1(n-1) + 2^0(n-0)\right]$

let $n-k=1$ ⇒ $k=n-1$

put $k=n-1$

$\left[2^{k-1}\left(\underbrace{n-k}_{1}+1\right)+2^{k-2}\left(\underbrace{n-k}_{1}+2\right)+\dots+2^1(n-1)+2^0(n)\right]$

$= 2^{n-2}\cdot 2 + 2^{n-3}\cdot 3 + 2^{n-4}\cdot 4 + \dots + 2^{n-n}\cdot n$

$T = 2^{n-1}(1) + 2^{n-2}(2) + 2^{n-3}(3) + \dots + 2^0(n)$

$2T = 2^n(1) + 2^{n-1}(2) + 2^{n-2}(3) + \dots + 2^1(n)$

$T = 2^n + 2^{n-1}(2-1) + 2^{n-2}(3-2) + \dots - 2^0(n)$

$= \left(2^n + 2^{n-1} + \dots + 2^1\right) - n$

$= \frac{2\left(2^n-1\right)}{(2-1)} - n$

$= 2^{n+1} - 2 - n$

$T(n) = O(2^n)$

Q. $T(n) = \begin{cases} 2 & \text{if } n=2 \\ n^{1/2}\,T(n^{1/2})+n & \text{if } n>2 \end{cases}$

can only be solved by substitution.

$\rightarrow T(n) = n^{1/2}\,T(n^{1/2}) + n$

$= n^{1/2}\left[(n^{1/2})^{1/2} \cdot T((n^{1/2})^{1/2}) + n^{1/2}\right] + n$

$= n^{1/2}\left[n^{1/4}\,T(n^{1/4}) + n^{1/2}\right] + n$

$= n^{\frac{1}{2}+\frac{1}{4}}\,T(n^{\frac{1}{4}}) + 2n$

$= n^{\frac{1}{2}+\frac{1}{4}+\frac{1}{8}+\cdots+\frac{1}{2^k}}\,T(n^{\frac{1}{2^k}}) + kn$

$n^{\frac{1}{2^k}} = 2$

$= n^{1-\frac{1}{2^k}}\,T(n^{\frac{1}{2^k}}) + kn$

$= \frac{n}{n^{\frac{1}{2^k}}}\,T(2) + kn$

$= \frac{n}{2} \cdot 2 + kn = n + kn = n + n\log\log n$

$T(n) = O(n \log\log n)$

$\frac{1}{2}+\frac{1}{4}+\frac{1}{8}+\cdots+\frac{1}{2^k} = \frac{(\frac{1}{2})(\frac{1}{2^k}-1)}{(\frac{-1}{2})}$

$= 1 - \frac{1}{2^k}$

$n^{\frac{1}{2^k}} = 2$

$k = \log\log n$

$\frac{1}{2^k}\log_2 n = \log_2 2 = 1$

$2^k = \log_2 n$

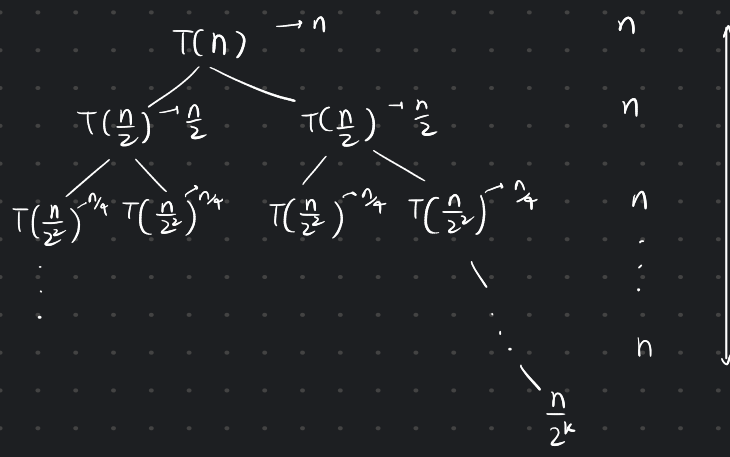
$k \log_2 2 = \log_2 \log_2 n$

$K = \log\log n$

Recursive tree method

Q. $T(n) = \begin{cases} 1 & n=1 \\ T(\frac{n}{2}) + T(\frac{n}{2}) + n & \text{if } n>1 \end{cases}$

problem of size n → problem of size $n/2$ + problem of size $n/2$ + n ← time taken for dividing problem into subproblems



Base condition, $T(1) = 1$

$\frac{n}{2^k} = 1$

$k = \lg n$

∴ # of levels = $\lg n$

effort at each = n

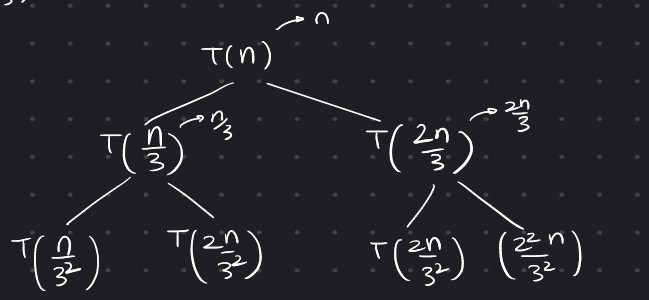
∴ $T(n) = O(n \lg n)$

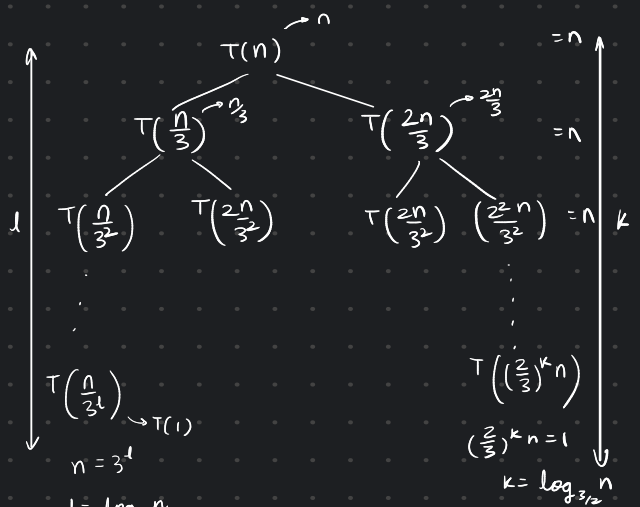
space complexity = height of the tree

space = $O(\lg n)$

Q. $T(n) = \begin{cases} 1 & \text{if } n=1 \\ T(\frac{n}{3}) + T(\frac{2n}{3}) + n & \text{if } n>1 \end{cases}$

→





$n = 3^l$

$l = \log_3 n$

$k = \log_{3/2} n$

$n(\log_{3/2} n) > n(\log_3 n)$

$\frac{2}{3^k} n = 1$

$2n = 3^k$

$n = 3^k$

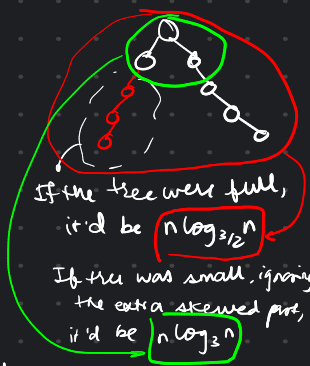
$k = \log_3 n$

$(\frac{2}{3})^k n = 1$

$n = (\frac{3}{2})^k$

$k = \log_{\frac{3}{2}} n$

Intuition:





Actual time lies betⁿ T_1 and T_2.

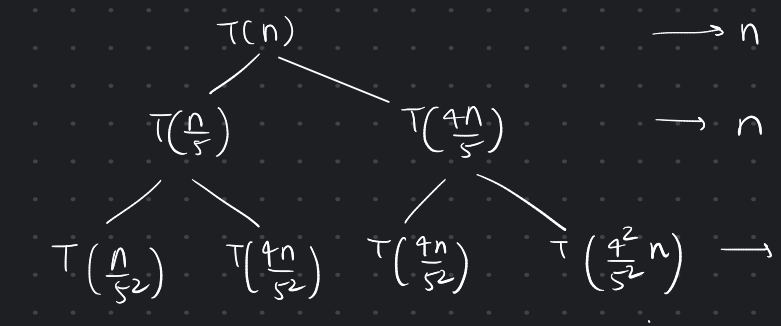
$T(n) \geq n(\log_3 n)$ $T(n) \leq n \log_{3/2} n$

$T(n) = \Omega(n \log_3 n)$ $T(n) = O(n \log_{3/2} n)$

$T = \Theta(n \log n)$ ← Base is ignored

Q. $T(n) = \begin{cases} 1 & n=1 \\ T(n/5) + T(4n/5) + n & n>1 \end{cases}$

→



Space complexity = max. depth

$= O(\log_{5/4} n)$

$= O(\log n)$

$T\left(\frac{n}{5^{k_2}}\right)$ $\qquad$ $T\left(\frac{4^{k_1}}{5^{k_1}} n\right) \longrightarrow n$

$n = 5^{k_2}$

$k_2 = \log_5 n$ $\qquad$ $k_1 = \log_{(5/4)} n$

$n \log_5 n \leq T(n) \leq n \log_{5/4} n$

$T(n) = \Omega(n \log_5 n)$ $\qquad$ $T(n) = O(n \log_{5/4} n)$

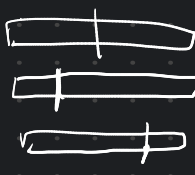
$T(n) = \Theta(n \log n)$

Q. $T(n) = \begin{cases} 1 & n=1 \\ T\left(\frac{n}{10}\right) + T\left(\frac{9n}{10}\right) + n & n>1 \end{cases}$

→

$T(n) = \Omega(n \log_{10} n)$, $O(n \log_{\frac{10}{9}} n)$, $\Theta(n \log n)$

If we divide problem in 2 subproblems in cases like the above,
we get $T(n) = \Theta(n \log n)$
space $= \Theta(\log n)$



however uneven the division is.

Q. $T(n) = \begin{cases} 1 & \text{if } n=1 \\ T\left(\frac{n}{10}\right) + T\left(\frac{7n}{10}\right) + n \end{cases}$ → In this case, $\frac{n}{10} + \frac{7n}{10} = \frac{8n}{10} \neq n$

→ $T(n)$ → n

The work done at every level changes.

$T\left(\frac{n}{10}\right)$ $T\left(\frac{7n}{10}\right)$ → $\frac{n}{10} + \frac{7n}{10} = \frac{8n}{10}$

$T\left(\frac{n}{10^2}\right)$ $T\left(\frac{7n}{10^2}\right)$ $T\left(\frac{7n}{10^2}\right)$ $T\left(\frac{7^2}{10^2}n\right)$ → $n\left[\frac{1}{10^2} + \frac{7}{10^2} + \frac{7}{10^2} + \frac{7^2}{10^2}\right] = \frac{64}{100}n = \frac{8^2}{10^2}n$

$T\left(\frac{n}{10^k}\right)$ $T\left(\frac{7^{k_1}}{10^{k_1}}n\right)$

$n = 10^k$ $k_1 = \log_{\frac{10}{7}} n$

$k = \log_{10} n$

$n + \frac{8n}{10} + \frac{8^2}{10^2}n + \cdots$ ($\log_{10} n$ + terms)

$n\left[\frac{\left(\frac{8}{10}\right)^{\log_{10} n + 1} - 1}{\left[\frac{8}{10} - 1\right]}\right]$ → 0 as $n \to \infty$

$= \Omega(n)$

$T(n) \leq \left(\frac{8}{10}\right)^0 n + \left(\frac{8}{10}\right)^1 n + \left(\frac{8}{10}\right)^2 n + \cdots + \left(\frac{4}{5}\right)^{\log_{\frac{10}{7}} n} n$

$\log_{\frac{10}{7}} n$ terms

$= n \cdot \frac{\left[\left(\frac{4}{5}\right)^{\log_{\frac{10}{7}} n + 1} - 1\right]}{\left[\frac{4}{5} - 1\right]}$ → 0 as $n \to \infty$

$= O(n)$

Space complexity $= \Theta(\log n)$

Q. $T(n) = \begin{cases} 1 & \text{if } n=1 \\ T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + n^2 & \text{if } n>1 \end{cases}$ → here division adds upto 1, but at each level, there's different work to add up the subproblems.

→ $T(n)$ → n^2

$T\left(\frac{n}{3}\right)$ $T\left(\frac{2n}{3}\right)$ → $\frac{n^2}{3^2} + \frac{2^2}{3^2}n^2 = \frac{5}{9}n^2$

$T\left(\frac{n}{3^2}\right)$ $T\left(\frac{2n}{3^2}\right)$ $T\left(\frac{2n}{3^2}\right)$ $T\left(\frac{2^2 n}{3^2}\right)$ → $\frac{1+4+4+16}{81}n^2 = \frac{25}{81}n^2$

$T\left(\frac{n}{3^k}\right)$

$\log_3 n$ levels

$T(n) = n^2\left[\frac{\left(\frac{5}{9}\right)^{\log_3 n} - 1}{\left[\frac{5}{9} - 1\right]}\right]$

$= \Omega(n^2)$

$T\left(\frac{n}{\left(\frac{3}{2}\right)^k}\right)$

$\log_{\frac{3}{2}} n$ levels

$T(n) = n^2\left[\frac{\left(\frac{5}{9}\right)^{\log_{\frac{3}{2}} n} - 1}{\left[\frac{5}{9} - 1\right]}\right]$

$= O(n^2)$

$= \Theta(n^2)$

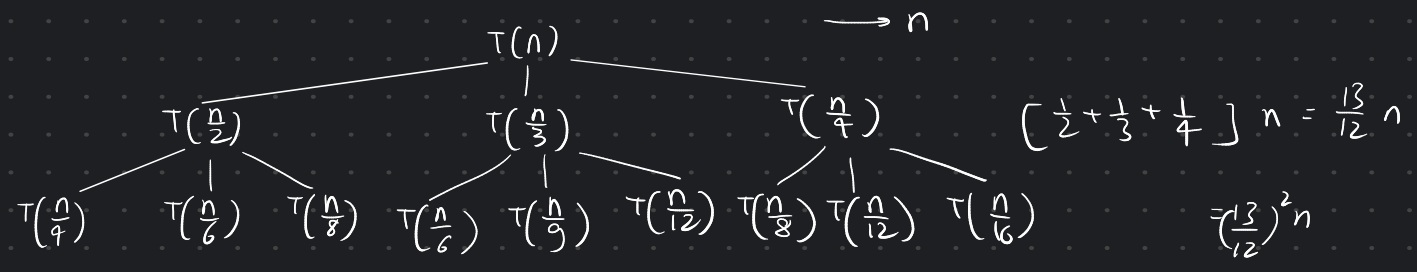
$\left[\left(\frac{5}{9}\right)^0 + \left(\frac{5}{9}\right)^1 + \left(\frac{5}{9}\right)^2 + \cdots\right] n$

$T(n) = \Theta(n^2)$

space $= \Theta(\log n)$

Q. $T(n) = \begin{cases} 1 \\ T(\frac{n}{2}) + T(\frac{n}{3}) + T(\frac{n}{4}) + n & \text{if } n>1 \end{cases}$

→



$\left[\frac{1}{2}+\frac{1}{3}+\frac{1}{4}\right] n = \frac{13}{12} n$

$= \left(\frac{13}{12}\right)^2 n$

$T\left(\frac{n}{2^k}\right)$

$k = \log_2 n$

$T\left(\frac{n}{4^m}\right)$

$m = \log_4 n$

$T(n) \le n\left[\left(\frac{13}{12}\right)^0 + \left(\frac{13}{12}\right)^1 + \cdots + \left(\frac{13}{12}\right)^{\log_2 n}\right]$

$\le n\left[\frac{\left(\frac{13}{12}\right)^{\log_2 n} - 1}{\frac{13}{12} - 1}\right]$

$\le n\, n^{\log_2\left(\frac{13}{12}\right)}$

$= O\left(n \cdot n^{\log_2\left(\frac{13}{12}\right)}\right)$

$T(n) \ge n\left[\left(\frac{13}{12}\right)^0 + \left(\frac{13}{12}\right)^1 + \cdots + \left(\frac{13}{12}\right)^{\log_4 n}\right]$

$\ge n\left[\frac{\left(\frac{13}{12}\right)^{\log_4 n + 1} - 1}{\frac{13}{12} - 1}\right]$

$\ge n\left(\frac{13}{12}\right)^{\log_4 n}$

$\ge n\, n^{\log_4\left(\frac{13}{12}\right)}$

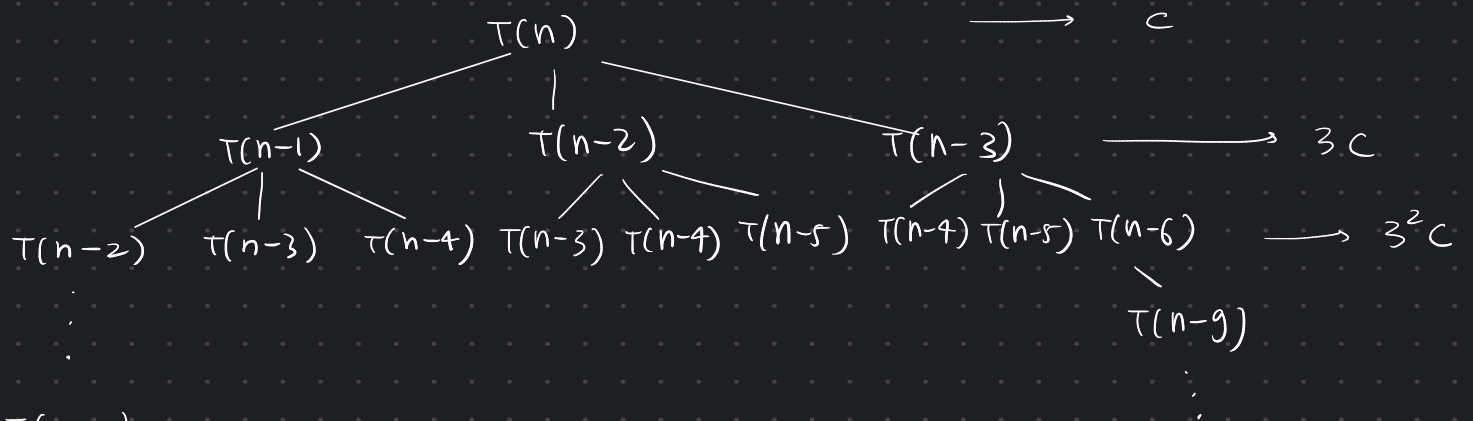
$= \Omega\left(n \cdot n^{\log_4 \frac{13}{12}}\right)$

✓ max depth

Space $= O(\log_2 n)$

Q. $T(n) = \begin{cases} 1 & \text{if } n = 1 \\ T(n-1) + T(n-2) + T(n-3) + c & \text{if } n \geq 1 \end{cases}$

→



$k = n-1$

$T(n) \leq \left[3^0 + 3^1 + \cdots + 3^{n-1}\right] c$

$\leq \left(\frac{3^n - 1}{3-1}\right) c$

$= O(3^n)$

$T(n) \geq \left[3^0 + 3^1 + \cdots + 3^{n/3}\right] c$

$\approx \Omega(3^n)$

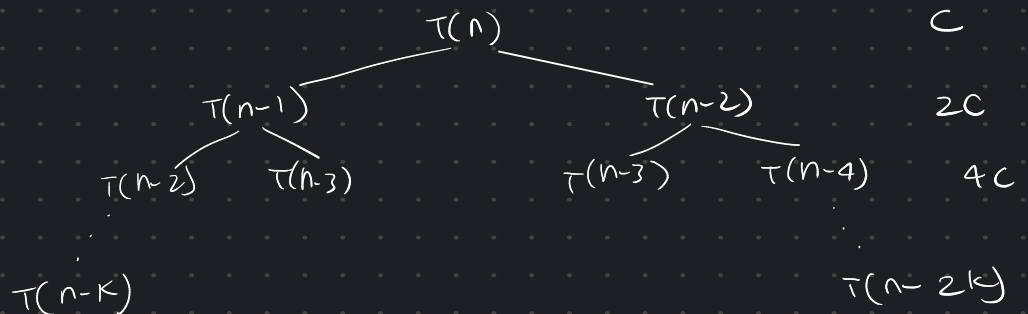
depths are almost-same on either sides.

$T(n) = O(3^n)$

space $= O(n)$

Q. $T(n) = \begin{cases} T(n-1) + T(n-2) + c & n > 1 \\ 1 & n = 1 \end{cases}$

→



$n - K = 1$

$K = n-1$

So $T(n) = c + 2c + 4c + 8c + \cdots$ $n-1$ terms

$= c\left[1 + 2 + 4 + 8 + \cdots\right)$

$= c \cdot \frac{2^{n-1} - 1}{2-1}$

$= O(2^n)$

Master theorem

Master theorem is only applicable for recurrence equations of type $T(n) = aT\left(\frac{n}{b}\right) + f(n)$,

$a \geq 1$, $b \geq 1$, a, b are constants. $f(n)$ is some function of n.

Let $T(n) = aT\left(\frac{n}{b}\right) + f(n)$, s.t. a and b are constants and $a \geq 1$, $b > 1$, then

Case 1: If $f(n) = O(n^{\log_b a - \epsilon})$, $\epsilon > 0$, then

$T(n) = \Theta(n^{\log_b a})$

Case 2: If $f(n) = \Omega(n^{\log_b a + \epsilon})$, $\epsilon > 0$, then

$T(n) = \Theta(f(n))$

Case 3: If $f(n) = \Theta(n^{\log_b a} (\log n)^k)$ where k is a constant, $k \geq 0$,

$T(n) = \Theta(n^{\log_b a} (\log n)^{k+1})$

Say $f(n) = n \log n$ $\quad n^{\log_b a} = n$.

$n \log n > n$, but not greater by a polynomial.

Here, we'll make them equal.

$n \log n \quad n(\log n)^{1=k} \quad \Theta(n(\log n)^{k+1}) = \Theta(n(\log n)^2)$

②

$f(n) \quad g(n)$

$n(\log n)^{10} > n$ But not polynomial greater.

$n(\log n)^{10} \quad n(\log n)^{10 \to k} \quad \Theta(n(\log n)^{11})$

③ $f(n) \quad n^{\log_b a}$

$n(\log n)^5 > n$ not poly. >

$n(\log n)^5 \quad \Theta(n(\log n)^6)$

④ $f(n) \quad n^{\log_b a}$

$n^2 \log n > n$

greater by poly. time

$\Theta(n^2 \log n)$

⑤ $f(n) \quad g(n)$

$n^3 > n^2$

poly. greater.

$\Theta(n^3)$

⑥ $f(n) \quad g(n)$

$n^7 \quad n^5$

$\Theta(n^5)$

⑦ $f(n) \quad g(n)$

$n \log n > n$

But NOT by poly.

$\Theta(n \log n (\log n)) = \Theta(n(\log n)^2)$

(simply multiply $\log n$)

$f(n)$ $n^{\log_b a}$

$n(\log n)^{10} > n$ not poly. greater

$\Theta(n \cdot (\log n)^{10} \cdot \log n) = \Theta(n \cdot (\log n)^{11})$

↓ multiply $\log n$ to greater one

$f(n)$ $g(n)$

n^2 n^2

↓

$n^2 (\log n)^0$ n^2

$\Theta(n^2 (\log n)^{0+1}) = \Theta(n^2 \log n)$

$f(n)$ $g(n)$

n^3 n^3

$\Theta(n^3 \log n)$

Q. $T(n) = 8T\left(\frac{n}{2}\right) + n^2$

→ $a=8, b=2, n^{\log_b a} = n^{\log_2 8}$

$f(n)$ $\quad$ $n^{\log_b a}$

$n^2 \quad < \quad n^3$ ↳ greater by poly.

$\Theta(n^3)$

Q. $T(n) = 2T\left(\frac{n}{2}\right) + n^2$

→ $a=2, b=2, \quad n^{\log_b a} = n^{\log_2 2} = n$

$f(n)$ $\quad$ $n^{\log_b a}$

$n^2 \quad > \quad n$

↳ greater by poly. $\quad \Theta(n^2)$

Q. $T(n) = 2T\left(\frac{n}{2}\right) + n$

$f(n)$ $\quad$ $n^{\log_b a}$

$n \quad\quad n$

$\Theta(n \log n)$

Q. $T(n) = 7T\left(\frac{n}{2}\right) + n^2$

$f(n)$ $\quad$ $n^{\log_b a}$

$n^2 \quad\quad n^{\log_2 7}$

$n^2 \quad < \quad n^{2.81}$

greater by poly.

$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$

Q. $T(n) = T\left(\frac{n}{2}\right) + n$

$f(n)$ $\quad$ $n^{\log_b a}$

$n \quad\quad n^0$

$n \quad > \quad 1$

↳ > by poly. $\quad \Theta(n)$

Q. $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$

→ $f(n)$ $\quad$ $n^{\log_b a}$

$n \log n \quad > \quad n$

↳ not > by poly.

$\Theta(n (\log n)^2)$

Q. $T(n) = 8T\left(\frac{n}{2}\right) + n^3 \log n$

→ $f(n)$ $\quad$ $n^{\log_b a}$

$n^3 \log n \quad\quad n^3$

$\Theta(n^3 (\log n)^2)$

Q. $f(n) = 2T\left(\frac{n}{2}\right) + n (\log n)^9$

→ $f(n)$ $\quad$ $n^{\log_b a}$

$n (\log n)^9 \quad\quad n$

$\Theta(n (\log n)^{10})$

Q. $T(n) = 2T\left(\frac{n}{2}\right) + n^2 \log n$

→ $f(n)$ $\quad$ $n^{\log_b a}$

$n^2 \log n \quad > \quad n$

↳ > by poly. $\quad \Theta(n^2 \log n)$

Q. $T(n) = 2^n T\left(\frac{n}{2}\right) + n^n$

→ ~~$f(n)$~~ $\quad$ ~~$n^{\log_b a}$~~

~~n^n~~ $\quad\quad$ ~~n^n~~

~~$\Theta(n^n \log n)$~~

For master theorem,

"a" should be a constant.

master theorem is not applicable.

use substitution.

Q. $T(n) = 0.5T\left(\frac{n}{2}\right) + n$

→ master theorem not applicable as $a < 1$.

****
***

Q. $T(n) = T(\sqrt{n}) + c$

→ Assume $n = 2^k$

$T(2^k) = T(2^{k/2} + c)$

Assume $T(2^k) = S(k)$

$S(k) = S\left(\frac{k}{2}\right) + c$ → master thm is applicable.

$f(k)$ $\quad$ $k^{\log_b a}$

$c \quad = \quad 1$

$\Theta(\log k)$

$S(k) = \Theta(\log k)$

$T(2^k) = \Theta(\log k)$

$n = 2^k$

$k = \log n$

$T(n) = \Theta(\log \log n)$

Q. $T(n) = 2T(\sqrt{n}) + \log_2 n$

→ let $n = 2^k$

$T(n) = 2T(2^{k/2}) + k$

let $T(2^k) = S(k)$

$S(k) = 2S(\frac{k}{2}) + k$

$f(k)$	$k^{\log_b a}$
k	k

$S(k) = \theta(k \log k)$

$T(2^k) = \theta(k \log k)$

$T(n) = \theta(\log n \log\log n)$

Q. $T(n) = T(\sqrt{n}) + c$. $T(2) = 1$

using substitution:

$T(n) = T(\sqrt{n}) + c$

$= T(n^{1/2}) + c$

$= T(n^{1/4}) + c + c$

$= T(n^{1/8}) + 3c$

$= T(n^{1/2^k}) + kc$

let $n^{\frac{1}{2^k}} = 2$

$\frac{1}{2^k} \log n = 1$

$\log n = 2^k$

$k = \log\log n$

$= T(2) + \log\log n \cdot c$

$T(n) = \theta(\log\log n)$

Using recurrence relations

Assume a function:

```
A(n) {
  if (n ≤ 1)
    return 1;
  else
    return n * A(n-1);
}
```

Recurrence relation: $A(n) = \begin{cases} 1 & n \le 1 \\ n \cdot A(n-1) & n > 1 \end{cases}$

Recurrence relation for time complexity: $T(n) = \begin{cases} O(1) & \text{if } n \le 1 \\ T(n-1) + c & \text{if } n > 1 \end{cases}$

↳ constant time for multiplicⁿ operaⁿ.

of multiplicⁿs for 1 funcⁿ call = 1 $\quad m(n) = \begin{cases} 0 & \text{if } n \le 1 \\ m(n-1) + 1 & \text{if } n > 1 \end{cases}$

```
A(n) {
  if (n ≤ 1) return n;
  else
    return A(n/3) + A(2n/3) + n;
}
```

For output: $v(n) = \begin{cases} n, & n \le 1 \\ v(\frac{n}{3}) + v(\frac{2n}{3}) + n, & n < 1 \end{cases}$

Time $\quad T(n) = \begin{cases} c & \text{if } n \le 1 \\ T(\frac{n}{3}) + T(\frac{2n}{3}) + c & n > 1 \end{cases}$

For addiⁿ operaⁿ (2 for each funcⁿ call): $add(n) = \begin{cases} 0 & \text{if } n \le 1 \\ add(\frac{n}{3}) + add(\frac{2n}{3}) + 2 & n > 1 \end{cases}$

```
f(n) {
  if (n==0 || n==1) return n;
  else
    return f(n-1) + f(n-2);
}
```

Value: $v(n) = \begin{cases} n & n=0 \mid n=1 \\ v(n-1) + v(n-2) & \text{otherwise} \end{cases}$

Time: $T(n) = \begin{cases} c & n=0 \mid n=1 \\ T(n-1) + T(n-2) + c & \text{otherwise} \end{cases}$